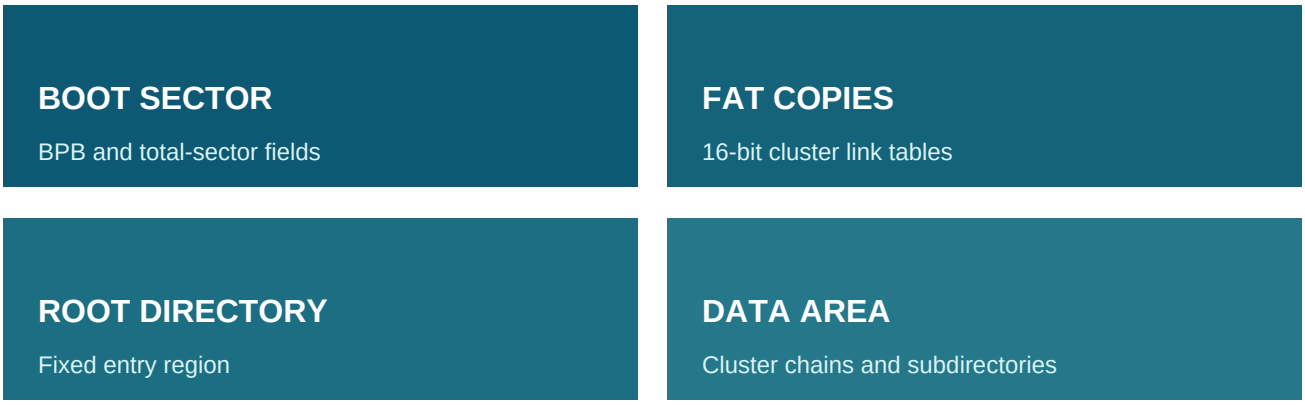
FAT16 - the DOS hard disk era

ORIGINAL FORMAT / HISTORY / ENGINEERING DESIGN

FAT16 extended the cluster-index field to sixteen bits and became a central disk format for DOS, Windows and a generation of removable storage.

This reference describes the original upstream disk format and its historical engineering rationale. The separate InfiltratorOS adapter is not the original specification. Historical facts, format rules and implementation limitations are separated; consult the named primary sources for field-level normative requirements.

ON-DISK STRUCTURE / CONCEPTUAL MAP



CHAPTERS

1. Historical chronology
2. Original design and on-disk structures
3. Allocation and directories
4. Integrity, validation and recovery
5. Design trade-offs and historical limitations
6. InfiltratorOS interpretation

FAT16 - the DOS hard disk era

HISTORICAL CHRONOLOGY

FAT16 grew from the original FAT12 architecture during the expansion from floppy-only microcomputers to larger fixed disks. Its sixteen-bit entries were adopted by DOS-era hard disk systems in the mid-1980s and remained common in DOS, Windows 3.x, Windows 9x and many cameras and embedded devices. The format is best understood as an evolutionary extension of the Microsoft FAT family rather than an entirely new on-disk model.

Later DOS and Windows releases expanded BPB fields and permitted larger partitions and cluster sizes. VFAT extensions allowed long filenames while preserving short-name compatibility. FAT32 ultimately removed the practical FAT16 cluster-count ceiling for larger disks, but FAT16 retained utility for boot partitions, small cards and older systems with simple firmware.

FAT16 - the DOS hard disk era

ORIGINAL DESIGN AND ON-DISK STRUCTURES

The volume begins with a boot sector containing a BPB and sometimes an extended BPB. Physical sector size, sectors per cluster, reserved-area length, FAT count, root directory entry count, sectors per FAT and total sectors determine all region offsets. The system area is followed by the FAT copies, a fixed root-entry region and the file-data area.

FAT16 entries occupy two bytes each, unlike FAT12’s packed twelve-bit pairs. They represent free space, successor clusters, bad clusters, reserved values and end-of-chain ranges. Cluster zero and one have special meaning. The FAT16 format is selected by computed data-cluster count, at least 4085 and below 65525, not by the string FAT16 in a boot sector.

FAT16 - the DOS hard disk era

ALLOCATION AND DIRECTORIES

Files use directory entries with a first-cluster reference and logical byte length. The allocator links each cluster to a successor in a FAT, so sequential file access follows the chain. One table can be copied to a second FAT, but the copies need not always be in sync after partial writes or faults. Allocation is a graph traversal with consistency checks, not a free-space bitmap scan.

The FAT16 root directory has a bounded number of fixed 32-byte entries. Subdirectories are ordinary cluster chains and can grow if free clusters exist. Names were originally DOS 8.3, with long names encoded through ordered and checksummed VFAT directory entries. Cluster size affects addressable volume size as well as wasted tail space in small files.

FAT16 - the DOS hard disk era

INTEGRITY, VALIDATION AND RECOVERY

FAT16 includes neither an original write-ahead journal nor end-to-end content checksums. A metadata update may alter a directory record, one or more FAT sectors, and file data; a power failure between writes can leave orphaned or multiply linked clusters. A safe checker validates both FAT copies, directory trees, file lengths, the cluster graph and out-of-range records before proposing repairs.

To discriminate FAT12, FAT16 and FAT32, first compute root-directory sectors, the first data sector, total data sectors and cluster count using the published algorithm. Do not rely on boot sector text labels. Write implementations must preserve unrelated packed BPB and reserved data, and may not reinterpret another family member because a user selected the wrong module.

FAT16 - the DOS hard disk era

DESIGN TRADE-OFFS AND HISTORICAL LIMITATIONS

Sixteen-bit entries are straightforward and small, but the number of available data clusters is bounded. Larger disk capacities require larger cluster sizes, increasing slack for small files. The root-directory limit is an independent problem on FAT16. The central allocation table can become a metadata hotspot and file fragmentation increases chain-walk costs.

FAT16’s widespread adoption stemmed from bootability, a simple interchange format and a very large software installed base. It lacks Unix ownership and link semantics, original-case Unicode name support without VFAT, native snapshots and crash-durable transactions. Recovery success is strongly dependent on the consistency of multiple pieces of metadata.

FAT16 - the DOS hard disk era

INFILTRATOROS INTERPRETATION

Our FAT16.so and optional kernel compatibility adapter are separate implementations of the FAT16 format, not replacements for its historical specification. FAT12 and FAT32 are distinct on-media variants with different entry encodings and layout rules. The identical high-level cluster-chain concept does not license reusing the wrong formatter or interpreting geometry from a guessed disk size.

Defragmentation and recovery capability must be established by module-specific tests and failure-injection evidence. A completed PDF build establishes documentation packaging only; it is not evidence that physical-media writes or repair paths are safe.

FAT16 - the DOS hard disk era

PRIMARY SOURCES & DOCUMENTATION

References are included to make the historical and technical account auditable. The source documents, published format specifications and implementation references take precedence over this explanatory guide where a detail differs.

1. Microsoft FAT Specification (historical normative on-media structure)

https://www.scs.stanford.edu/~zyedidia/docs/_other/fat.pdf

2. Linux kernel VFAT notes and compatibility options

https://docs.kernel.org/filesystems/vfat.html

Implementation boundary: the installed InfiltratorOS .so, the optional Linux kernel .ko, and the source-first installer are distinct products. The original filesystem design is documented above; claims about the adapter’s qualification require its own source, tests and release evidence.

FAT16 / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. Boot/BPB: sector size, cluster size and total-volume sector count.
2. RootEntCnt: bounded root table independent of normal subdirectories.
3. FATSz16: size of each 16-bit cluster-chain table.
4. FAT[0] and FAT[1]: reserved entries and initial media state.
5. FAT[n]: 16-bit next cluster, free, bad or end-of-chain marker.
6. FirstDataSector: reserved + FAT sectors + root directory sectors.
7. Directory entry: high-level 32-byte DOS record and optional VFAT extensions.
8. Data cluster n: start of data region + (n-2) * cluster size.

FAT16 / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. Mid-1980s: FAT16 addresses the growth of DOS hard disks.

2. 1990s: extended BPB and larger clusters permit larger volumes.

3. Mid-1990s: VFAT long names extend directory conventions.

4. 1996 onward: FAT32 addresses FAT16’s capacity and allocation-granularity limits.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

FAT16 / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Check computed data cluster count is 4085 to 65524.

2. Do not mistake the fixed root directory for a cluster chain.

3. Treat the two FAT copies as potentially divergent after faults.

4. Validate every directory file-size and first-cluster reference.

5. Keep DOS short-name and VFAT directory records consistent.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

FAT16 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Parse the BIOS Parameter Block to derive reserved area, FAT sizes, root-directory sectors, first data sector and the total addressable cluster population.

2. FAT16 has a fixed root-directory region whose size derives from RootEntCnt and bytes per sector. This differs from FAT32’s relocatable root cluster.

3. Classify the actual format by data-cluster count rather than by filesystem type text, disk size alone, or the partition-type byte.

4. Each FAT16 link occupies a 16-bit little-endian slot. Interpret free entries, reserved ranges, bad-cluster markers and end-of-chain markers separately.

5. Walk directories in 32-byte entries, handling volume labels, file attributes and VFAT long-filename entry chains without confusing them with normal files.

6. Convert a valid data cluster number into an offset from the calculated first data sector; cluster numbering begins at two.

7. Read each data chain in logical order, limit content by the file’s length, and bound traversals to avoid loops from corrupt allocation entries.

8. Recognise that sector size and cluster-size parameters vary. Assumptions about a fixed 512-byte sector or one-sector cluster should not be embedded in readers.

FAT16 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Verify that FAT capacity, cluster count and the boot-record geometry describe a coherent volume before reading or writing clusters.

2. Treat the fixed root region as directory storage, not a chain in the data area. Subdirectories use data clusters and may grow as needed.

3. When two FAT copies disagree, preserve both and use structural consistency evidence rather than assuming either is definitively correct.

4. Reject chain loops, invalid cluster references, cross-links and clusters beyond the actual data region. Account separately for clean end-of-chain markers.

5. Check long-name ordinal ordering and short-name checksums before associating Unicode fragments with a directory entry.

6. Directory timestamps use DOS date/time encoding and limited precision; they are not Unix timestamps and cannot encode every date/time combination.

7. An allocator must update FAT links, directory metadata and the free-space state coherently or an interruption can produce orphan chains and cross-links.

8. Near FAT12/FAT16 boundaries, re-evaluate the computed cluster count after a proposed reformat or resize; a cosmetic type label cannot make a structurally invalid volume safe.